

Homework 4 (CSCI-B 504)

Kushal Pokharel

November 2025

1 Problem 1

Decrypt the ciphertext encrypted by Merkle-Hellman knapsack system. Download the problem from `knapsack_cryptosystem_problem.txt`. Also please download the matrix from `Knapsack_problem_matrix.txt`

1.1 Approach

First, I used the **primality testing** to get prime numbers just greater than the given numbers. I incremented the numbers by 1 in each round and see if that number passes Rabin's primality test, if it passes for 50 numbers, I can claim with high confidence that those numbers are prime.

After that, the decryption process was straightforward. First, finding the **inverse of W in M** and **multiplying that inverse by the array y**, to get the permuted private key array. I then **sort this private key array as it must be superincreasing**. I have asserted that condition in the code as well. This sorting also gives me the permutation of the bits that I need to arrange later to get back the true plaintext. Finally, I got the bits by using the super-increasing private key array and permuted these bits as described above, and got the number corresponding to that bit sequence. **I expected to recover 14 digits** (7 plaintext characters were given as a hint, each character = row and col = $7 \times 2 = 14$) to be recovered, and if it was less than that, I appended 0 in the front. This would only happen if the first k characters were space (0,0) characters. Finally, I got those seven plaintext characters by looking up the matrix's row and column as given by that number.

1.2 Decrypted text

MiraMesa is a community and neighborhood in the city of San Diego, California. The city-recognized MiraMesa Community Plan Area is roughly bounded by Interstate 15 on the east, Interstate 805 on the west, the Los Penasquitos Canyon on the north and Marine Corps Air Station Miramar on the south.[2] Most of the community plan area is referred to as Mira Mesa; the community plan area also includes the neighborhoods of Sorrento Valley and SorrentoMesa.

2 Problem 2

Merkle-Hellman knapsack system is not secure and attacks have been found. Of course, like any other research problems, research on improvements to Merkle-Hellman knapsack public key systems have been continuously proposed and the attacks have been continuously found, even upto very recently. a) Please discuss/prove one attack to the initial Merkle-Hellman public key system. b) please discuss/prove one improvement to Merkle-Hellman knapsack public key system and one or two attacks to the initial or improved Merkle-Hellman knapsack public key systems.

2.1 Shamir's attack

From [1], it first states that given the public key array, a trapdoor pair of natural numbers M and W is to be found such that $W \cdot a_i$ is a superincreasing sequence. Since M needs to be greater than $\sum_i a_i$, these private key elements are quite smaller than M , which means that public key elements are large as well. Let's define $U = W^{-1} \bmod M$. Our goal is to find this pair of trapdoor integers. As shown in Figure 1, we plot U vs $U * a_i$, where a_i is the public key. and to be consistent with the algorithm, W here means U in our case, because W is already defined and U is W^{-1} . **The key observation lies in the fact that $W a_i$ gives us potential secret key and we know our secret key is much less than M . Hence, our true secret key would lie in one of the minima in this sawtooth graph seen below.**

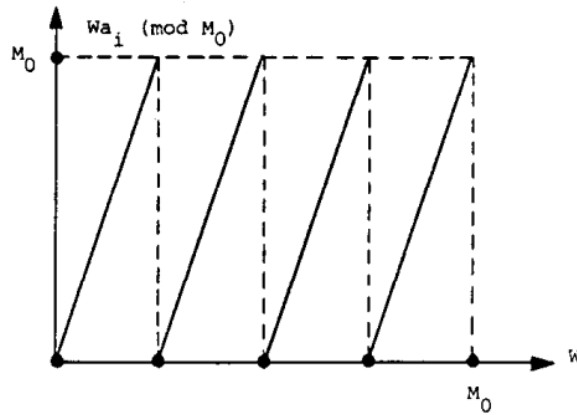


Fig. 1.

Figure 1: Sawtooth plot with different values of $U a_i$ against U

But there are still too many minima here to try out the values of U . Note that this is only the graph for a_i . We also have a few other public keys, a_i , and can plot a similar graph to observe those minima as well. Shamir observed that the minima of the two curves were very close to each other. This closeness greatly reduces the number of U 's but still doesn't uniquely identify it. With some interval calculation, the paper shows that we have to analyse about 4 curves in some conditions.

However, the paper discusses two problem of not knowing M and how to accumulate those points. The author suggests dividing both of these axes on 1 by M and plotting it again, where the number of successive minima remains the same but their distances decrease as shown below:

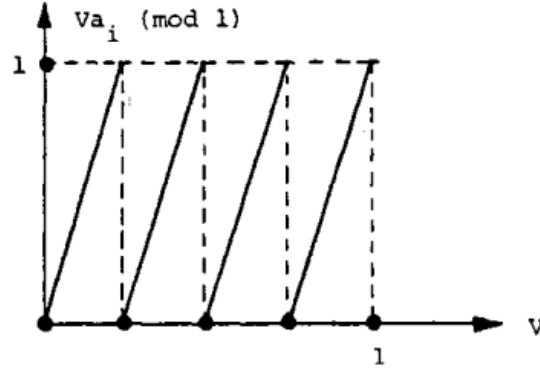


Fig. 2.

Figure 2: Figure 1 Divide both axes by M

Finally, we can represent the accumulation points like this. Let's say p^{th} minima in a_1 , q^{th} minima in a_2 , r^{th} minima in a_3 are near, their relationship can be defined as below. Where the differences between the minima are tightly bound.

$$\begin{aligned} p, q, r, \dots, \text{integers}, \quad & 1 \leq p \leq a_1 - 1, \\ -\epsilon_2 \leq p/a_1 - q/a_2 \leq \epsilon'_2, \quad & 1 \leq q \leq a_2 - 1, \\ -\epsilon_3 \leq p/a_1 - r/a_3 \leq \epsilon'_3, \quad & 1 \leq r \leq a_3 - 1, \\ & \vdots \quad \vdots \end{aligned}$$

where the ϵ_i and ϵ'_i represent the allowable deviations to the right and to the left of p/a_1 , respectively. By multiplying each double inequality by its denominators,

we get the equivalent system

$$\begin{aligned}
p, q, r, \dots, \text{integers, } & 1 \leq p \leq a_1 - 1, \\
-\delta_2 \leq pa_2 - qa_1 \leq \delta'_2, & 1 \leq q \leq a_2 - 1, \\
-\delta_3 \leq pa_3 - ra_1 \leq \delta'_3, & 1 \leq r \leq a_3 - 1, \\
& \vdots \\
& \vdots
\end{aligned}$$

As said in the paper, this system of inequalities, where we are trying to **minimise two variables in the presence of a third variable as a modulus, can be solved with the continued fraction algorithm**. Furthermore, Lenstra's integer programming can be applied to a more general case. The paper also states that with tight bounds, even the permutation version of the algorithm is crackable.

2.2 Improvement to the Subset sum based cryptosystem - Iterated Merkle-Hellman Knapsack

The attack above was based on the fact that the public key (array) and private key (super-increasing array) were related by a constant W . Hence, an improvement was proposed where an extra layer of array was added β^1 before β^2 - the public key was calculated.

The steps looked like this:

1. Start with a super increasing array w_i
2. Apply $(W1, M1)$ to get an array $\beta^1 = W1 * w_i \bmod M1$.
3. Apply $(W2, M2)$ to get an array $\beta^2 = W2 * \beta^1 \bmod M2$.

This was done to break the linearity between the public and private array, but further lattice-based reduction algorithms could easily break this improved cryptosystem with high confidence. I will try to explain how those reductions work in next section

2.3 Designing lattice and LLL reduction

For context, a Lattice is a set of points over Z^n . A lattice is uniquely identified by its bases. All integer linear combinations of those bases span that lattice. Lenstra Lenstra Lovasz reduction (LLL) algorithm is a reduction technique that can find bases for a lattice given some generating points, i.e if I am given 10 n-dimensional points randomly, I can apply LLL to get d-bases that can generate those 10 points and more. Also, LLL can be applied to get better bases from any given base for a lattice.

A lattice could have multiple bases. A base is better than the other base if it is shorter and if the base vectors are more perpendicular to each other. LLL algorithm applies Graham-Schmidt orthogonalisation to get better base vectors

at each iteration. Having said that, we can only get so much better bases. If we were able to get the best base vector, we would be able to find the shortest vector in the lattice, as the shortest vector is one of the base vectors (directly implied from the fact that a lattice can't have any lattice point inside a parallelepiped/fundamental region). Hence, LLL tries to approximate the Shortest Vector Problem (SVP), which is an NP-hard problem, in polynomial time.

2.3.1 LLL in Merkle-Hellman

Merkle-Hellman is a low-density knapsack problem. This example shows how to break the low-density subset problem, which Merkle-Hellman is based on.

Problem Statement. Let $M := 2^{4n^2}$ (big number). Given $\vec{a} \in \mathbb{Z}^n$, such that each entry a_i (public key) is sampled independently, uniformly at random from $U := \{\frac{1}{2}M, \frac{1}{2}(M+1), \dots, M\}$, and a target S computed as $S := \langle \vec{a}, \vec{x} \rangle$, recover $\vec{x}$.

Note that finding $\vec{x}$ is recovering the message.

Consider the lattice generated by the following basis.

$$B = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \\ 2^n \cdot S & -2^n a_1 & -2^n a_2 & \dots & -2^n a_n \end{pmatrix} \quad (1)$$

The lattice $\mathcal{L}(B)$ contains the vector $\vec{b}_0 + \sum_{i=1}^n x_i \vec{b}_i = (\vec{x}, 0)^T$ (by construction). The lattice is specifically designed in such a way so that when linear combination with $\vec{x}$ we get a small value, but with any other linear combination, we get a very large value.

Because $\vec{x}$ is a binary vector,

$$\|(\vec{x}, 0)^T\|_2 = \|\vec{x}\|_2 \leq \sqrt{n}$$

We can use the LLL algorithm to find a lattice vector of length at most

$$\|\vec{x}\|_2 \cdot 2^{n/2} \ll 2^n$$

LLL may not end with exactly $\vec{x}$, but it can be shown that it will always end with a multiple of $\vec{x}$.

3 Problem 3

1. Decrypt the ciphertext encrypted by a simple one time cipher (i.e., a specific stream cipher) using PGM as a random number generator and the random numbers generated act as a keystream.

- (a) Let $M := 95 * 95 * 95$; seed = 1000.
- (b) Suppose plaintext string $\vec{x} = x_1x_2 \dots x_n$ with each $x_i < M$.
- (c) keystream $\vec{k} = k_1k_2 \dots k_n$, where $k_1 = \text{PGM}(\text{seed}), \dots, k_i = \text{PGM}(\text{seed} + i - 1), \dots, k_n = \text{PGM}(\text{seed} + n - 1)$.
- (d) ciphertext string $\vec{y} = y_1y_2 \dots y_n$, where $y_i = (x_i + k_i) \bmod M$.
- (e) The plaintext string $\vec{x} = x_1x_2 \dots x_n$ is obtained by encoding an english text $E = e_1e_2 \dots$ as follows:
 - i. There are 95 characters given in a table and each character has its index (0 to 94).
 - ii. (NOTE: the beginning character is space and the table should be treated as one dimension.)
 - iii. Group english text into blocks of three characters as $e_1e_2e_3, e_4e_5e_6, e_7e_8e_9 \dots$
 - iv. Form $x_1, x_2, \dots$ as follows: $x_1 = (\text{index of } e_1) \cdot 95^2 + (\text{index of } e_2) \cdot 95^1 + (\text{index of } e_3)$,

$$x_2 = (\text{index of } e_4) \cdot 95^2 + (\text{index of } e_5) \cdot 95^1 + (\text{index of } e_6),$$

and so on...

2. The PGM logarithmic signatures and the ciphertext can be downloaded from PGM_logs.txt and PGM_ciphertext.txt. In addition, please also download ...
3. When you decrypt, please remember to change negative modular into positive.
4. The permutations in logarithmic signatures are expressed in standard notation.

3.1 Approach

I followed all the conventions from the slides, which include the order of combining the permutation, the order of peeling off the permutation for the inverse of β . With this, I couldn't get the right plaintext, as the plaintext below is most probably not correct.

One doubt I have is in the format of the ciphertext given, whose encoding wasn't properly mentioned in the PDF. I used the same technique to retrieve the numerical ciphertext as it was described for the plaintext.

One thing I am certain of is: My technique to get the β permutation is correct. Because in the last, after applying all the β^{-1} , **I get the identity**

permutation, which confirms the correct β decoding. However, maybe with incorrect encoding, I couldn't correctly get the final English plaintext.

Here is the log of one block decryption of my code:

```
Ciphertext block z<\: 814970
Encrypting plaintext: 1728
Alpha indices: [8, 1, 3, 2, 0, 0, 0, 0]
Final combined permutation: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Alpha indices: [8, 1, 3, 2, 0, 0, 0, 0]
Beta indices: [5, 6, 7, 0, 1, 1, 0, 0]
Final encrypted value: 35975
For key plaintext 1728 ciphertext 814970: Encrypted key value: 35975
For ciphertext block z<\: Decrypted value: 778995
Index at position 0: 86 778995
v
Index at position 1: 29 2845
=
Index at position 2: 90 90
z
Full decrypted plaintext: kCM=CiUc*s=#uoN9d mDD[jChK4i;3Apgei4'hV%}V,gI/A;0s*9}U*)PJ!1-!IZQ79]e70Z2JD9G5u/(qZ0
'fGSL!L'm\s] OI.48ngarF~_VvB24H@%NH0t/c_a;+++ M]S4"$>9@ $onYE.m6=6wAL(1%7R$" .Np\ e6}$0ppZ?rZj.
P xfrD7n~IyKTQ]_40A\g&[<PUihm7em0b3<1W'ubpTOFg;&o*4k_}{?x"d)@Z.l(*C2z_w>8v0Zt1+K"ec<
$yYEZl':p*u d+ohwPF?n|2pZ|@f8?$+4Ww'1U{u:Qv.>MiczT[xeDyPIL{z^U}NT0BZ;a,wR;WrU$;;{x;;t/55>
Mb0'~~1VPF:ZF0,Rv+i=tvQz nsFojVMDLn~Usi_?(c;WbfH#\k|>P'OGbi|M:;J*A^Sx{WxMi(dy%)xoVmJU4-RrF5E
odw!Jt!8]h8T,>s/aQ'XH?v%R[m2~cn0jIg|T,-5/ 93'J!Q T[4v/J(|)5~r7>?#a2G)%+oltxQ<eyhgb)o$RH_7uc2
;ir' IPFmvX;CavP%6KI$4U\'/2&Xb1,k'iuPizxLEYDj~dep8BMguf\z*TPh/72<YBa#
hT81q00#3Ur,MCafY_tAt+]*U#cBP)'27yCk*0. "sNT7" gymqKc.Y@' '?%J(di#6[2dC9+9_:.! .Kt
ae3@9QT(1AbE3dS@>'1BVCJ~L=k|2j}_z}WJ;xrM0FfaZTe1/~;{zyCe#H<ZQ[|2vS<{.Pm%$1oWB<x,'Gz'!16k"K=
T.@jHpf6|@A#1Ne<p=<NoE+dAp c#"c*c|#pjSh[&g[3=hmBwjF\N(z3-jNuWx"4'iW)SiC}Q"L4H(T"W'[jkj4$H|/
i+~f &MuzOFcwz\+17V91Bd,+[ZGVsLL9uf~_pd'>U19&p_J<]8pcejyT'VhEH)[_A041y{u}MAOHsR|3Ukv<9:|At\B
\\U ?'rbbrGM9kL2F}j.n~n1Fu|\F[tjJLSj2t'(y%P!pZ\\r#!pk:yI5UR4dY;aB
~[S% CTH'.){Sd2{-%nGb{up;w5-Usl\PC*,*u$lo'>\5jA9BAYpm:6|}q~tg>9P47#1ZJp<$
R5v--/{w~v&lF]e"?*v<U3Esn'yoEi?I_|/A~TJLE1'SGq~oI,EIB6SCJ(8t)t4TV'$I&72'1^DcV#-bfOWGdC&?gM_V
?miS_'Gf.<vH'TV:0SrG;;j{NkIw xQ%;E~:HKy?v2;Cad6Y&Y4z)Ur.JxktiMmuxrLyR2_:YzR^QjHf_LPLL91z'p=
(+Gf=#dZ>fOdy3Zpx8oI"x4MT?B*3g8Hfp4/w7$-)Mj/04z+A=UH:Mu:<V/]-'48N?8DZn>_Hv1soG}e)TUD\A.WT(r
S%b't4GW\&t}gam82#e'U*ffXq0o.eL]1QRfU3m>"'x6z,k/>m)SLq) t1{i7K7.*_HL#MmkoA#T<EGR
uxAeApV'CuBLr>VoTDyh~<+}den9{0'~'0#Cy+D?Y}LeV#+5bcHYEf?XH=b(MJ'%'_Q]LFZ(CL')*X+Gm/O+$'>bCd~
```

Figure 3: Log from program for one block of ciphertext decryption

3.2 Decrypted plaintext

kCM=CiUc*s=#uoN9d mDD[jChK4i;3Apgei4'hV%}V,gI/A;0s*9}U*)PJ!1-!IZQ79]e70Z2JD9G5u/(qZ0
'fGSL!L'm\s] OI.48ngarF~_VvB24H@%NH0t/c_a;+++ M]S4"\$>9@ \$onYE.m6=6wAL(1%7R\$" .Np\ e6}\$0ppZ?rZj.
P xfrD7n~IyKTQ]_40A\g&[<PUihm7em0b3<1W'ubpTOFg;&o*4k_}{?x"d)@Z.l(*C2z_w>8v0Zt1+K"ec<
\$yYEZl':p*u d+ohwPF?n|2pZ|@f8?\$+4Ww'1U{u:Qv.>MiczT[xeDyPIL{z^U}NT0BZ;a,wR;WrU\$;;{x;;t/55>
Mb0'~~1VPF:ZF0,Rv+i=tvQz nsFojVMDLn~Usi_?(c;WbfH#\k|>P'OGbi|M:;J*A^Sx{WxMi(dy%)xoVmJU4-RrF5E
odw!Jt!8]h8T,>s/aQ'XH?v%R[m2~cn0jIg|T,-5/ 93'J!Q T[4v/J(|)5~r7>?#a2G)%+oltxQ<eyhgb)o\$RH_7uc2
;ir' IPFmvX;CavP%6KI\$4U\'/2&Xb1,k'iuPizxLEYDj~dep8BMguf\z*TPh/72<YBa#
hT81q00#3Ur,MCafY_tAt+]*U#cBP)'27yCk*0. "sNT7" gymqKc.Y@' '?%J(di#6[2dC9+9_:.! .Kt
ae3@9QT(1AbE3dS@>'1BVCJ~L=k|2j}_z}WJ;xrM0FfaZTe1/~;{zyCe#H<ZQ[|2vS<{.Pm%\$1oWB<x,'Gz'!16k"K=
T.@jHpf6|@A#1Ne<p=<NoE+dAp c#"c*c|#pjSh[&g[3=hmBwjF\N(z3-jNuWx"4'iW)SiC}Q"L4H(T"W'[jkj4\$H|/
i+~f &MuzOFcwz\+17V91Bd,+[ZGVsLL9uf~_pd'>U19&p_J<]8pcejyT'VhEH)[_A041y{u}MAOHsR|3Ukv<9:|At\B
\\U ?'rbbrGM9kL2F}j.n~n1Fu|\F[tjJLSj2t'(y%P!pZ\\r#!pk:yI5UR4dY;aB
~[S% CTH'.){Sd2{-%nGb{up;w5-Usl\PC*,*u\$lo'>\5jA9BAYpm:6|}q~tg>9P47#1ZJp<\$
R5v--/{w~v&lF]e"?*v<U3Esn'yoEi?I_|/A~TJLE1'SGq~oI,EIB6SCJ(8t)t4TV'\$I&72'1^DcV#-bfOWGdC&?gM_V
?miS_'Gf.<vH'TV:0SrG;;j{NkIw xQ%;E~:HKy?v2;Cad6Y&Y4z)Ur.JxktiMmuxrLyR2_:YzR^QjHf_LPLL91z'p=
(+Gf=#dZ>fOdy3Zpx8oI"x4MT?B*3g8Hfp4/w7\$-)Mj/04z+A=UH:Mu:<V/]-'48N?8DZn>_Hv1soG}e)TUD\A.WT(r
S%b't4GW\&t}gam82#e'U*ffXq0o.eL]1QRfU3m>"'x6z,k/>m)SLq) t1{i7K7.*_HL#MmkoA#T<EGR
uxAeApV'CuBLr>VoTDyh~<+}den9{0'~'0#Cy+D?Y}LeV#+5bcHYEf?XH=b(MJ'%'_Q]LFZ(CL')*X+Gm/O+\$'>bCd~

References

- [1] A. Shamir, "A polynomial time algorithm for breaking the basic merkle-hellman cryptosystem," in *Proceedings of the 23rd Annual Symposium on*

Foundations of Computer Science, ser. SFCS '82. USA: IEEE Computer Society, 1982, p. 145–152.